ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PAPER 1 · PRACTICE D · MARKING GUIDELINE

MARKS: 120

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming paper 1 · practice d.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Code is not marked strictly on syntax: give marks for correct logic and structure. Accept other correct answers, variable names and wording.

SECTION A

QUESTION 1

1.1 B ✓
1.2 A ✓
1.3 D ✓
1.4 C ✓
1.5 C ✓
1.6 A ✓
1.7 D ✓
1.8 B ✓
1.9 C ✓
1.10 D ✓

(10)

[10]

QUESTION 2

2.1 True ✓
2.2 False ✓
2.3 True ✓
2.4 False ✓
2.5 True ✓
2.6 False ✓
2.7 True ✓
2.8 True ✓
2.9 False ✓
2.10 True ✓

(10)

[10]

QUESTION 3

3.1 register ✓
3.2 binary ✓
3.3 unit test ✓
3.4 float ✓
3.5 upper() ✓

3.6 nested ✓
3.7 tuple ✓
3.8 object ✓
3.9 commit ✓
3.10 sprints ✓

(10)

[10]

QUESTION 4

4.1 D ✓
4.2 I ✓
4.3 G ✓
4.4 A ✓
4.5 H ✓
4.6 E ✓
4.7 J ✓
4.8 B ✓
4.9 K ✓
4.10 C ✓

(10)

[10]

TOTAL SECTION A: [40]

SECTION B

QUESTION 5

5.1 Arithmetic logic unit (ALU) ✓ – does the calculations and logical comparisons ✓. Control unit (CU) ✓ – fetches and decodes instructions and controls the other parts ✓. Registers ✓ – very fast storage that holds the instructions and data being used ✓. (Accept cache memory as a part with its function.) (6)

5.2.1 $45 = 32 + 8 + 4 + 1$ (or repeated division by 2) ✓ $= 101101$ ✓ (2)

5.2.2 $2\text{F} = 2 \times 16 + 15$ ✓ $= 47$ ✓ (2)

5.3 Waterfall: the phases are done one after the other, each finished before the next starts ✓; changes later in the project are difficult and the client sees the product only at the end ✓. Agile: the work is done in short cycles (sprints), each delivering a working part ✓; the client gives feedback often and changes are easy to include ✓. (4)

5.4.1 15 ✓✓ (5 × 3) (2)

5.4.2 2 times ✓ (1)

5.5 A branch is a separate copy of the code in a repository where changes can be made ✓ without affecting the main (working) version until they are tested and merged ✓. (2)

5.6 git commit saves a snapshot of the changes in the local repository on your computer ✓; git push sends (uploads) those commits to the remote repository on GitHub ✓. (2)

5.7 Any TWO ✓✓: a backup if the computer is lost or damaged; files can be reached from any computer; team members can work together; version history is kept. (2)

[23]

QUESTION 6

6.1 It catches the error that occurs when the input cannot be changed into a number ✓, so the program shows a message instead of crashing ✓. (2)

6.2 Please enter a number. ✓ (1)

6.3 It ends the function immediately ✓, so the rest of the report is not printed when there are no readings (which would cause errors such as min() of an empty list or division by zero) ✓. (2)

6.4 It makes a new list named close ✓ that contains only the readings smaller than 20 cm ✓ (a list comprehension). (2)

6.5 Readings: 4 ✓ / Closest: 12.5 ✓ / Average: 30.0 ✓ / Too close: 2 ✓ (4)

6.6 So that the user can type a or A (q or Q, r or R) and the choice is still recognised ✓. (1)

6.7 def with the correct name ✓; input with the question ✓; test for "Y" (with upper()) ✓; empty the list with clear() (or del readings[:]) ✓; message ✓ (5)

```
def clear_readings():
    answer = input("Are you sure? (Y/N) ").upper()
    if answer == "Y":
        readings.clear()
        print("All readings cleared.")
```

[17]

QUESTION 7

7.1 speeds = ✓ [120, 80, 150, 95, 60] ✓ (2)

7.2 print("Highest:", max(speeds)) ✓ and print("Lowest:", min(speeds)) ✓ (2)

7.3 speeds.sort(reverse=True) ✓ (or sorted(speeds, reverse=True)) and print(speeds) ✓ (2)

7.4 def average(values): ✓; total with sum() or a loop ✓; divided by len(values) ✓; return the result ✓ (4)

```
def average(values):
    return sum(values) / len(values)
```

7.5 Input of the code ✓; parts = code.split("-") ✓; display parts[1] as the year ✓; if code.startswith("RB") (or parts[0] == "RB") ✓; "Valid code" ✓; else "Invalid code" ✓ (6)

```
code = input("Serial code: ")
parts = code.split("-")
print("Year:", parts[1])
if code.startswith("RB"):
    print("Valid code")
```

```
else:
    print("Invalid code")
```

7.6 Counter set to 0 ✓; for loop over speeds ✓; if speed > 100 ✓; counter increased ✓; display after the loop ✓ with a message ✓ (6)

```
count = 0
for s in speeds:
    if s > 100:
        count += 1
print(count, "speeds are above 100 r/min")
```

[22]

QUESTION 8

8.1 Inheritance is when a new class (subclass / child) is based on an existing class (parent) ✓ and automatically gets its attributes and methods, which it can extend or change ✓. (2)

8.2 An attribute is data (a variable) that belongs to an object ✓, e.g. name or max_rpm ✓. A method is a function that belongs to a class and describes what an object can do ✓, e.g. set_speed() ✓. (4)

8.3.1 __init__ ✓ (1)

8.3.2 Speed out of range ✓ / 0 ✓ (2)

8.4 class ServoMotor(Motor): ✓; constructor with the parameters ✓; super().__init__(name, max_rpm) (or Motor.__init__) ✓✓; self.angle = 0 ✓; def turn_to(self, angle): ✓; if 0 <= angle <= 180 ✓; self.angle = angle ✓; else "Angle out of range" ✓ (9)

```
class ServoMotor(Motor):
    def __init__(self, name, max_rpm):
        super().__init__(name, max_rpm)
        self.angle = 0

    def turn_to(self, angle):
        if 0 <= angle <= 180:
            self.angle = angle
        else:
            print("Angle out of range")
```

[18]

TOTAL SECTION B: [80]

TOTAL: 120

END OF MARKING GUIDELINE